

## Low Vulnerability

### L1: Reflective XSS

| Matrix Calculation     |             | CVSS Score                                                                                          | Risk |
|------------------------|-------------|-----------------------------------------------------------------------------------------------------|------|
| Impact                 | Medium      | 4.8                                                                                                 | Low  |
| Likelihood             | Low         |                                                                                                     |      |
| CVSSv4.0 Attack Vector |             | <a href="#">AV:N/AC:L/AT:N/PR:L/UI:A/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N</a>                              |      |
| MITRE ATT&CK           | Techniques  | <a href="#">T1189: Drive-by Compromise</a>                                                          |      |
|                        | Mitigations | <a href="#">M0818: Validate Program Inputs</a><br><a href="#">M1021: Restrict Web-Based Content</a> |      |
| Affected Scope         |             |                                                                                                     |      |

### Description

An attacker can craft a malicious URL that contains JavaScript code that will be reflected the users browser when they click the URL.

### Business Impact

Attackers can abuse this vulnerability to execute arbitrary JavaScript code in victim browsers by distributing malicious links through phishing attempts, social engineering, and compromised websites. This can be abused to hijack session cookies or send malicious alerts and messages to other users.

### Steps to Reproduce

Step 1: Go to **WEBSITE** on a web browser

Step 2: Inject a malicious XSS payload into a URL parameter (e.g.,  
[https://WEBSITE/search?q=<script>alert\('XSS'\)</script>](https://WEBSITE/search?q=<script>alert('XSS')</script>))

Step 3: Send the crafted URL to a victim

Step 4: When the victim clicks the malicious URL the script will be executed

## Remediations

**Team-XX** recommends the following:

1. Properly sanitize all user input before storing or passing any data to users. Sanitization functions are present in all programming languages that will encapsulate and prevent the execution of any malicious user input.
2. Utilize a web application firewall to block dangerous requests and common injection patterns.
3. Utilize a strict Content Security Policy to restrict JavaScript sources.
4. Mark session cookie configurations as *HttpOnly*, *Secure*, and *SameSite=Strict* to protect cookies from exfiltration.

## Reference

[Mozilla - Secure Cookie Configurations](#)

[OWASP - XSS Attacks](#)

[Portswigger - Cross Site Scripting Prevention](#)

## L1: Lack of anti-CSRF Tokens

| Matrix Calculation     |             | CVSS Score                                                                                        | Risk |
|------------------------|-------------|---------------------------------------------------------------------------------------------------|------|
| Impact                 | Medium      | 5.1                                                                                               | Low  |
| Likelihood             | Low         |                                                                                                   |      |
| CVSSv4.0 Attack Vector |             | <a href="#">AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N</a>                            |      |
| MITRE ATT&CK           | Techniques  | <a href="#">T1078: Valid Accounts</a><br><a href="#">T1190: Exploit Public Facing Application</a> |      |
|                        | Mitigations | <a href="#">M1054: Software Configuration</a>                                                     |      |
| Affected Scope         |             |                                                                                                   |      |

### Description

The affected web applications do not implement CSRF protection. Without anti-CSRF tokens, an attacker can trick an authenticated victim to make unintended requests to the server. These requests can maliciously modify data, such as user passwords, in the server.

### Business Impact

If a privileged user is tricked by a threat actor into sending requests to the server to modify critical data, the data integrity of APT's network will be compromised.

### Steps to Reproduce

- Step 1: Go to **WEBSITE** on a web browser.
- Step 2: intercept any post request that modifies data.
- Step 3: Observe that it does not use any CSRF token.

### Remediations

**Team-XX** recommends the following:

1. Implement anti-CSRF tokens, using unique and random tokens for every request.
2. Validate all anti-CSRF tokens on the server side before processing any request.

## Reference

[OWASP - Cross Site Request Forgery](#)

## L1: SMTP Nonenforced DKIM and DMARC

| Matrix Calculation     |             | CVSS Score                                                                                       | Risk |
|------------------------|-------------|--------------------------------------------------------------------------------------------------|------|
| Impact                 | Medium      | 2.3                                                                                              | Low  |
| Likelihood             | Low         |                                                                                                  |      |
| CVSSv4.0 Attack Vector |             | <a href="#">AV:N/AC:H/AT:N/PR:L/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N</a>                           |      |
| MITRE<br>ATT&CK        | Techniques  | <a href="#">T1534: Internal Spearphishing</a><br><a href="#">T1598: Phishing for Information</a> |      |
|                        | Mitigations | <a href="#">M1054: Software Configuration</a>                                                    |      |

|                |  |
|----------------|--|
| Affected Scope |  |
|----------------|--|

## Description

The SMTP server is missing a DMARC and DKIM record. Unenforced DMARC allows an attacker to spoof the domain in phishing attacks by allowing unauthorized use of the domain in the Friendly-From address of email messages. Unenforced DKIM means that verification of senders is not performed on the SMTP server, enabling users to more easily perform phishing campaigns on ATP employees.

## Business Impact

Attackers can abuse this misconfiguration to perform domain spoofing in targeted phishing campaigns on ATP employees. These emails will not be verified as coming from internal ATP employees but will appear to be coming from the ATP domain. This will make phishing attempts much harder to detect and will likely result in increased employee interaction with malicious, phishing emails.

## Steps to Reproduce

Step 1: `swaks -ehlo user@domain -from name@domain -to user@domain -server smpt.server.local --h-From "Name" <name@domain> --attach-type text/html --attach-body @Mail-template.html`

## Remediations

**Team-XX** recommends the following:

1. Enforce DMARC by adding a record to the domain and specifying the enforcement policy to quarantine or reject emails containing a mismatch between sender and from, which is likely to be a domain spoofing attempt.
2. Enforce DKIM which will verify domain identities when sending emails.
3. Implement email quarantining and filtering to block malicious emails and phishing attempts from being delivered to employee inboxes.

## Reference

[RackSpace - Create a DMARC Policy](#)

[RackSpace - Enable DKIM](#)

[Higherlogic - SPF, DKIM, DMARC Email Authentication](#)

